SecureShare: A DevSecOps Security Engineering Sprint

CYC386 — Secure Software Design and Development — Midterm Lab Exam — Final Project Report

Syed Zurriat Hussain (FA23-BCT-041) · Muhammad Ahmadd (FA23-BCT-020) · Junaid (FA23-BCT-057)

COMSATS University Islamabad, Department of Computer Science | Fall 2026 | 2026-10-30

Executive Summary

This report documents a one-week DevSecOps security engineering sprint for CYC386 (Secure Software Design and Development), in which a three-person team built **SecureShare**, a Flask file-sharing web application, first as an intentionally vulnerable baseline and then hardened it against five mandatory vulnerability classes — Insecure Direct Object Reference (IDOR), Cross-Site Request Forgery (CSRF), Clickjacking, OS Command Injection, and a native-code stack Buffer Overflow. The team performed Protection Needs Elicitation (PNE) with explicit CIA-triad mapping, built a full STRIDE threat model with two attack trees and CVSS v3.1 risk scoring, demonstrated and then remediated each vulnerability with reproducible proof-of-concept exploits, and verified every fix with automated regression tests, SAST (Bandit, Semgrep), dependency auditing (pip-audit), and DAST (OWASP ZAP). A GitHub Actions CI/CD pipeline enforces these gates, failing the build on any High/Critical finding. All five mandatory vulnerability classes were remediated at their root cause (not merely mitigated), one additional CSP weakness surfaced by DAST was fixed in the same sprint, and seven known CVEs in a transitive dependency were resolved. The result is a working, documented, attack-resistant application with a complete audit trail from protection need through threat, fix, and test.

1. Introduction

1.1 Context and Mission

Modern software teams are expected to ship secure code under real time pressure, with security built into the development lifecycle rather than bolted on afterward — the core premise of DevSecOps. This sprint simulates that reality: a three-person engineering team was handed a one-week window to take an application from a documented, exploitable state to a verified, defensible one, following the same Protection Needs → Threat Model → Fix → Test traceability chain a real security review would demand.

1.2 Team

Member	Registration No.	Role
Syed Zurriat Hussain	FA23-BCT-041	Lead developer; threat modeling, IDOR/ Buffer Overflow/SAST remediation
Muhammad Ahmadd	FA23-BCT-020	Co-developer; CSRF/Clickjacking/Command Injection remediation, CI/CD pipeline, DAST
Junaid	FA23-BCT-057	Co-developer; documentation, Jira board, CI/ CD support, dependency hygiene

(See §9, Individual Contribution Statement, for the detailed breakdown and git commit evidence.)

1.3 Project Theme

File Sharing / Collaboration platform (Flask + SQLite), one of the CDF-recommended themes, chosen because it naturally exposes all five mandatory vulnerability classes on realistic attack surfaces: object references for shared files (IDOR), state-changing file actions (CSRF), an embeddable web UI (Clickjacking), a server-side export/convert feature shelling out to the OS (Command Injection), and a native C metadata-parsing helper (Buffer Overflow).

1.4 Document Map

This report synthesizes four companion documents, each going into far greater depth on its topic:

Companion document	Covers
`docs/PNE_Report.md`	Full asset inventory, stakeholder analysis, CIA mapping
`docs/THREAT_MODEL.pdf`	Full DFDs, STRIDE table, attack trees, CVSS vectors, risk register
`docs/SECURITY_IMPLEMENTATION.md`	Full vulnerable/fixed code, exploit transcripts, verification
`docs/SECURITY_TEST_REPORT.md`	Full SAST/DAST/dependency tool output, before/after

2. System Architecture

SecureShare is a single-process Flask 3 application backed by SQLite, with one native C helper process invoked per file-view request.

```
SecureApp-Sprint-ZurriatAhmad/
├── app/
│   ├── app.py            Flask routes, auth, authorization, security headers
│   ├── seed_demo.py       Demo account/data seeding (alice, bob)
│   ├── templates/         Jinja2 templates (login, register, dashboard, upload,
file view)
│   └── static/style.css   External stylesheet (CSP-compatible, no inline styles)
├── native/
│   ├── parser.c           Buffer-overflow-vulnerable metadata helper (kept as
"before" artifact)
│   ├── parser_fixed.c      Remediated helper (bounds-checked)
│   └── Makefile            Three build targets: vulnerable, fixed, ASan-
instrumented
├── tests/                 pytest regression suite, one file per vulnerability
class
├── docs/                   This report and its three companion documents
├── Screenshots/             12 numbered PoC/verification screenshots
└── .github/workflows/      CI/CD: build → test → SAST → DAST
```

Request flow: a user's browser talks to Flask over HTTP(S); Flask authenticates via a signed session cookie, authorizes every file access through a single centralized helper (`get_owned_file_or_403()`), reads and writes SQLite (`users` , `files` tables) and the filesystem (`app/uploads/` , `app/exports/`), and — only when a file is viewed — shells out via `subprocess` (no shell interpretation) to the native parser helper to extract a title field from a small custom metadata header. Every response passes through a single `after_request` hook that attaches the application's security headers (CSP, X-Frame-Options, etc.) regardless of which route produced it.

See `docs/THREAT_MODEL.md` §1-§2 for the full Context and Level-1 Data Flow Diagrams this architecture is modeled on.

---

3. Protection Needs Elicitation — Summary

Full detail: `docs/PNE_Report.md`. Assets, stakeholders, and ten protection needs (PN1-PN10) were elicited by walking each stakeholder's goals against each asset and by directly targeting the five mandatory vulnerability classes as known failure modes to design against. Each protection need maps explicitly to the CIA triad:

Protection Need	C	I	A	Addressed By
PN1 — file access restricted to its owner	✅	✅	—	`fix/idor`
PN2 — state-changing actions require deliberate, same-origin intent	—	✅	✅	`fix/csrf`
PN3 — pages not embeddable for UI redressing	✅	✅	—	`fix/clickjacking`
PN4 — no user input reaches a shell	✅	✅	✅	`fix/command-injection`
PN5 — native code bounds-checks all copies	✅	✅	✅	`fix/buffer-overflow`
PN6 — credentials/ sessions protected	✅	✅	—	Werkzeug password hashing, `HttpOnly` / `SameSite` cookies
PN7 — no known-vulnerable dependencies ship	✅	✅	✅	`fix/dependency-werkzeug`
PN8 — CI/CD blocks critical findings	✅	✅	✅	`.github/workflows/ci-cd.yml`
PN9 — authorization centralized	✅	✅	—	`get_owned_file_or_403()`
PN10 — malformed input fails safe, not catastrophically	—	—	✅	Upload-time length validation, bounded native parsing

This mapping is the backbone of the entire sprint: every fix below traces back to one or more of these ten rows.

4. Threat Model — Summary

Full detail (DFDs, full STRIDE table, both attack trees, CVSS justifications, residual risk): `docs/THREAT_MODEL.pdf`. A systematic STRIDE pass was applied to every process, data store, and data flow in the Level-1 DFD, surfacing five threats (T1-T5) that map 1:1 onto the five mandatory vulnerability classes, scored with CVSS v3.1 against the **vulnerable-baseline** state:

ID	Threat	CVSS v3.1 Base Score	Severity
T1	Command Injection (`/file/<id>/export`)	**9.9**	Critical
T5	Buffer Overflow (native parser)	**7.1**	High
T2	IDOR (`/file/<id>` , `/file/<id>/download`)	**6.5**	Medium-High
T3	CSRF (`/file/<id>/delete`)	**6.1**	Medium
T4	Clickjacking (all responses)	**5.7**	Medium

Two attack trees were built for the highest-impact goals: **(1) Remote Code Execution on the SecureShare host**, reachable via either the Command Injection path (T1, demonstrated to full completion) or the Buffer Overflow path (T5, demonstrated as a reliable crash/DoS primitive with AddressSanitizer confirming the exact memory-safety defect); and **(2) Unauthorized disclosure or destruction of another user's file**, reachable via direct IDOR (T2) or CSRF-forced deletion (T3, optionally chained with Clickjacking T4 to also disguise the malicious page's UI). Both trees are reproduced in full, with ASCII diagrams, in `docs/THREAT_MODEL.pdf` §4.

5. Detailed Remediation

This section summarizes each of the five mandatory vulnerabilities: root cause, exploit, fix, and verification. Full code listings, complete exploit transcripts, and all twelve PoC screenshots are in `docs/SECURITY_IMPLEMENTATION.md`; representative screenshots are reproduced below.

5.1 IDOR — CWE-639 (CVSS 6.5)

Root cause: `view_file()`, `download_file()` and `delete_file()` queried the `files` table by `id` alone, with no check that the requesting user owned that row.

Exploit: a second, unrelated user (`bob`) navigated directly to `/file/1` — an ID belonging to `alice` — and received her full file content and title, confirmed both via the browser and via `curl` with bob's session cookie downloading alice's confidential payroll notes outright.

Fix (`fix/idor`, commit `2f068c3`): a single centralized `get_owned_file_or_403(db, file_id, user)` helper now backs every file-touching route; any request for a file the caller does not own returns HTTP 403 and logs a warning.

Verification: manual re-test now returns 403 (Figure 1); `tests/test_idor.py` (3 tests) passes.

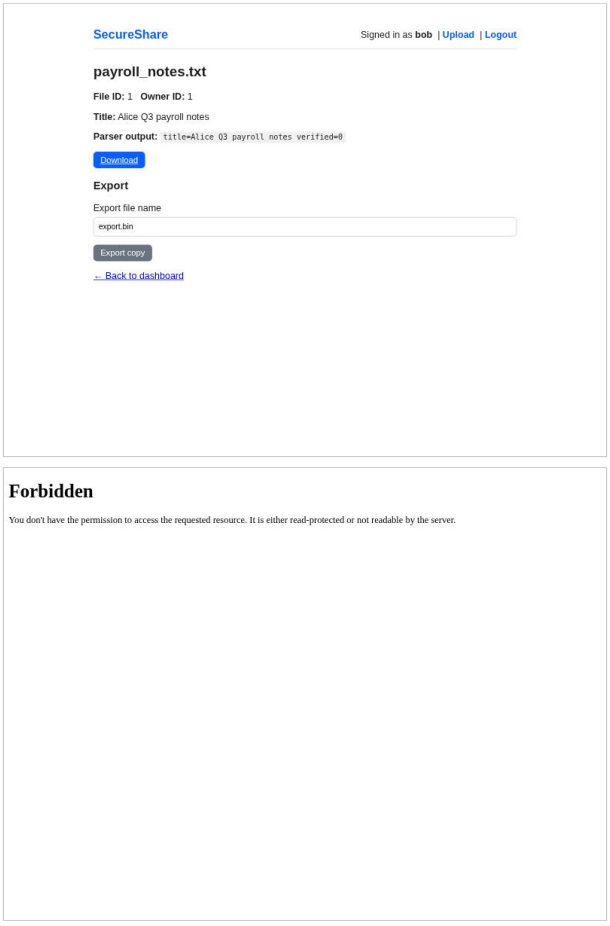


Figure 1. IDOR before (bob reads alice's file) / after (403 Forbidden).

5.2 CSRF — CWE-352 (CVSS 6.1)

Root cause: `POST /file/<id>/delete` relied solely on the session cookie for authorization, with no anti-CSRF token.

Exploit: while `alice` had a live session, her browser was navigated to an unrelated third-party page containing a single auto-submitting hidden `<form>` targeting the delete endpoint. Her file was deleted instantly, with no click and no confirmation — confirmed by her dashboard showing "No files yet." immediately after the page loaded.

Fix (`fix/csrf`, commit `5711e12`): Flask-WTF `CSRFProtect` enabled app-wide; every POST form now carries a hidden, session-bound `csrf_token`; `SESSION_COOKIE_SAMESITE=Lax` and `HttpOnly` added as defense-in-depth.

Verification: the identical attacker page now triggers "400 Bad Request: The CSRF token is missing." and the target file survives (Figure 2); `tests/test_csrf.py` (2 tests) passes.

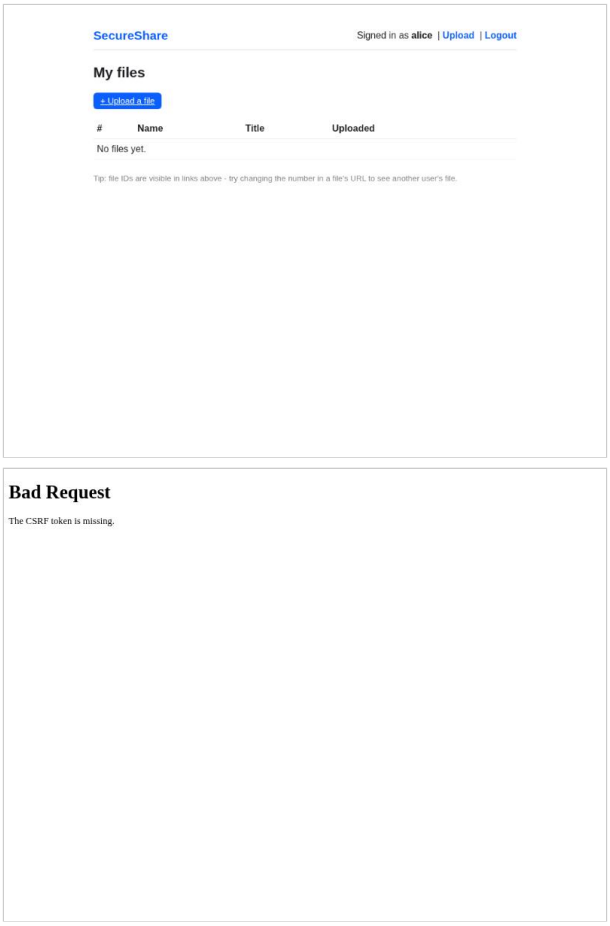


Figure 2. CSRF before (file silently deleted) / after (400, token missing).

5.3 Clickjacking — CWE-1021 (CVSS 5.7)

Root cause: no response carried `X-Frame-Options` or a `Content-Security-Policy`, so the application could be rendered inside a third-party `<iframe>`.

Exploit: a decoy "Win a free gift card" page overlaid a near-transparent iframe of SecureShare's dashboard on top of a fake button, rendering fully and interactively with no browser refusal.

Fix (`fix/clickjacking`, commit `ef97471`; hardened further in `chore/csp-hardening`, commit `3100566`): an `after_request` hook adds `X-Frame-Options: DENY` and a full `Content-Security-`

`Policy` (every directive explicitly locked to `'self'`, including `frame-ancestors 'none'`) to every response.

Verification: the identical attacker page now renders an empty iframe (Figure 3); `tests/test_clickjacking.py` (2 tests) passes; `curl -I` confirms both headers on every route.

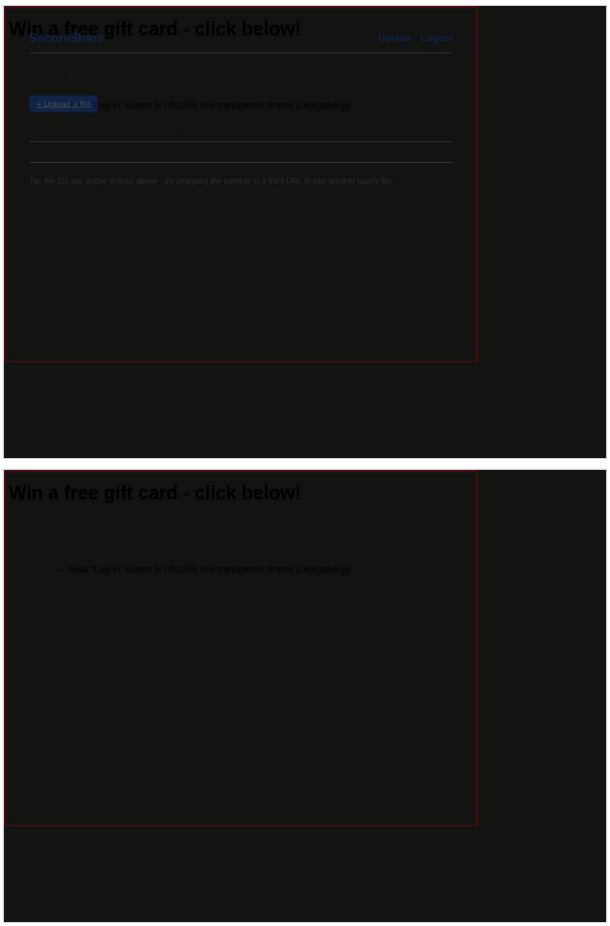


Figure 3. Clickjacking before (fully framed) / after (empty iframe, refused).

5.4 Command Injection — CWE-78 (CVSS 9.9, Critical)

Root cause: `export_file()` built a shell command string with `f"cp '{src}' '{dst_pattern}'"` and ran it via `subprocess.run(cmd, shell=True)`, with `dst_pattern` derived directly from user input.

Exploit: submitting `export_name = x'; id > /tmp/cmdi_proof.txt; echo pwned > /tmp/cmdi_marker.txt; echo '` broke out of the single-quoted shell string and ran `id` as a second, fully attacker-chosen command with the web server's own OS privileges, writing arbitrary files to `/tmp` — the most severe finding in the sprint.

Fix (`fix/command-injection`, commit `fab7c8a`): the shell call was removed entirely in favor of `shutil.copyfile()` (a direct OS call, no shell to inject into), plus a strict allow-list regex on `export_name` as defense-in-depth.

Verification: the identical payload is now rejected client-side with "Invalid export name..." and no proof file is ever created (Figure 4); `tests/test_command_injection.py` (2 tests) passes; Bandit's High-severity `B602` finding drops to zero.

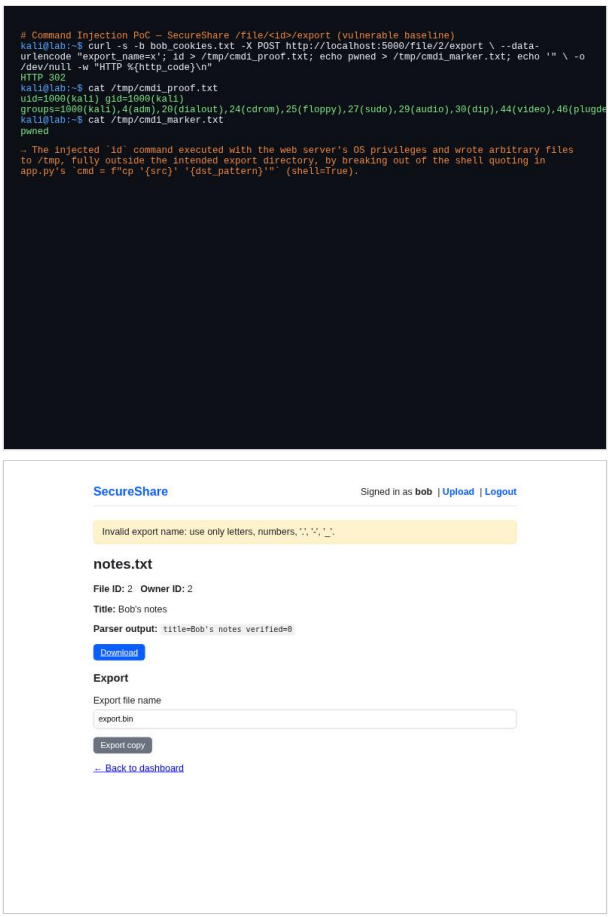


Figure 4. Command Injection before (id executed on host) / after (rejected by allow-list).

5.5 Buffer Overflow — CWE-121 (CVSS 7.1, High)

Root cause: `native/parser.c` copied an attacker-influenced `title` string into a fixed 64-byte stack buffer with `strcpy()`, with no bounds check, built without stack-protection mitigations for the lab demonstration.

Exploit: a 100-byte title reliably crashed the helper process (`SIGSEGV`, exit 139); AddressSanitizer pinpointed the exact defect — `stack-buffer-overflow` in `strcpy()` at `parser.c:48`, confirming the overflow reaches adjacent stack data (the `verified` field).

Fix (`fix/buffer-overflow`, commit `c6afce7`): `native/parser_fixed.c` replaces `strcpy()` with a length-checked `snprintf()` that rejects (rather than silently truncates) oversized input, built with `-fstack-protector-strong -D_FORTIFY_SOURCE=2 -pie`; the web layer additionally rejects an over-long title at `/upload` before native code is ever invoked.

Verification: the identical 100-byte input is now safely rejected (exit 2, "title field too long") with zero crashes, confirmed against both a plain rebuild and an ASan-instrumented rebuild (Figure 5); `tests/test_buffer_overflow.py` (2 tests) passes.

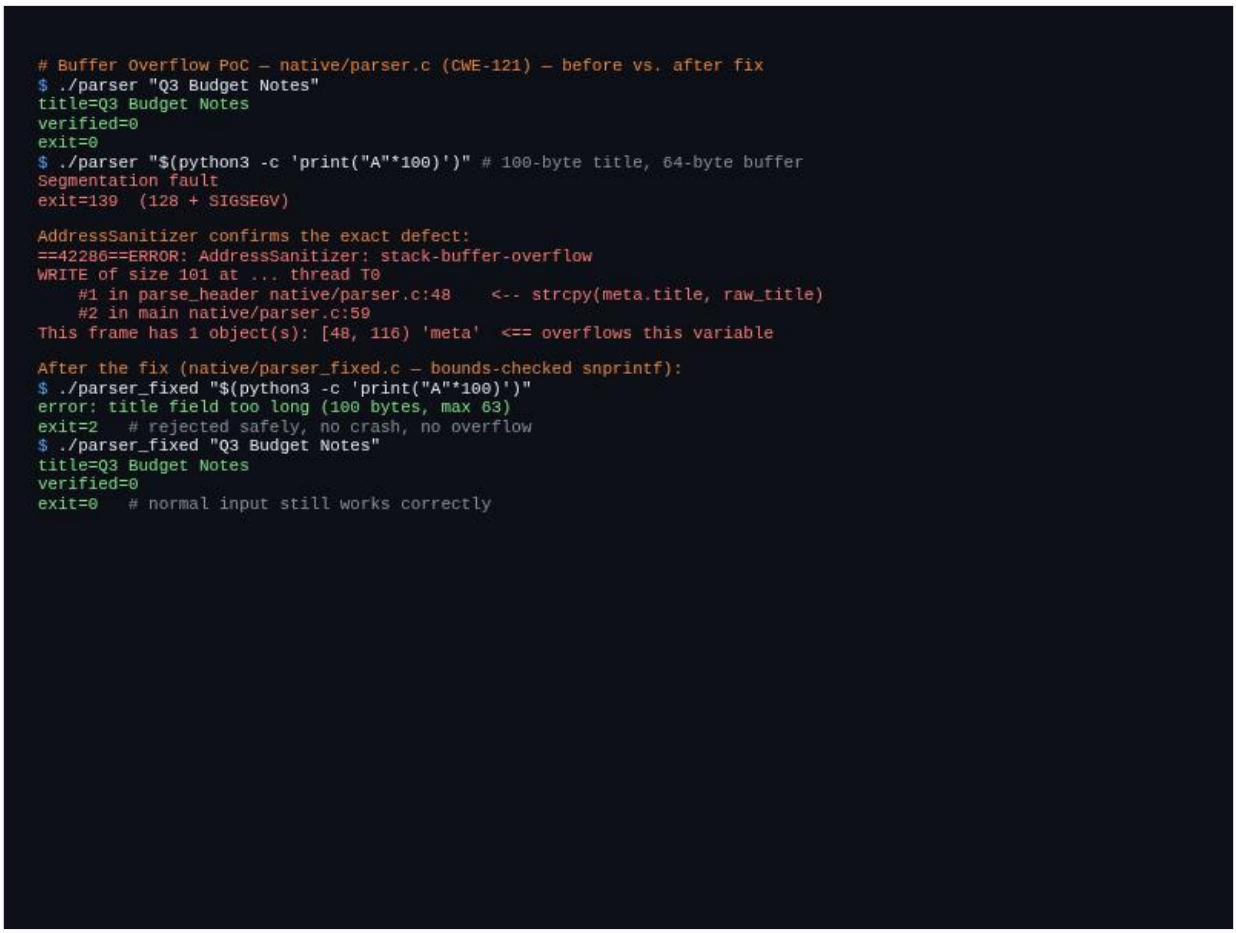


Figure 5. Buffer Overflow: plain crash + AddressSanitizer report (left) vs. safe rejection by the fixed binary (right).

6. Testing Evidence

Full tool output and methodology: `docs/SECURITY_TEST_REPORT.md`. Every fix was verified at up to four independent layers:

6.1 Unit / Regression Tests

```
$ python -m pytest tests/ -v
11 passed in 2.70s
```

Two tests per vulnerability class (attack blocked + legitimate flow still works), plus one extra IDOR ownership control.

6.2 SAST — Bandit & Semgrep

Tool	Before (`vulnerable-baseline`)	After (`secure-release`)
Bandit	1 High (`B602` , shell=True injection), 3 Low	**0 High**, 2 Low (reviewed, justified)
Semgrep	7 findings, all ERROR severity	**0 findings** (2 reviewed false positives triaged with inline justification)

CI hard gate (`bandit -r app/ -lll -iii` , `semgrep --severity=ERROR --error`) exits 0 against the fixed code and 1 (would fail the build) against the vulnerable baseline — reproduced and confirmed for this report.

6.3 Dependency Audit — pip-audit

7 known CVEs in `Werkzeug==3.1.3` (including `CVE-2025-66221` , directly relevant since the app uses `send_from_directory`) → **0** after bumping to `Werkzeug>=3.1.9` (`fix/dependency-werkzeug` , commit `57479f9`).

6.4 DAST — OWASP ZAP

A spider + active scan was run against both the live vulnerable baseline (port 5001) and the live fixed application (port 5000):

Finding category	Before	After
Missing Anti-clickjacking Header	3	**0**
CSP Header Not Set	5	**0**
Absence of Anti-CSRF Tokens	3	**0**
Cookie without SameSite Attribute	1	**0**

ZAP independently corroborated the Clickjacking and CSRF fixes at the network layer. It also surfaced a **new, bonus** finding: once a CSP existed, ZAP flagged that it left most directives undefined (29 Medium findings — "CSP: Failure to Define Directive with No Fallback", wildcard and `unsafe-inline` equivalents). This was triaged and fixed in the same sprint (`chore/csp-hardening`): page CSS was extracted to an external stylesheet so every CSP directive could be locked to `'self'` with zero `unsafe-inline` , verified with no CSP violations in the browser console.

6.5 CI/CD Pipeline

`.github/workflows/ci-cd.yml` runs three jobs — `build-and-test` → `sast` → `dast` — on every push/PR to `main` / `develop`. The `sast` job's exact commands were reproduced locally against both git tags: they exit non-zero (**fail the build**) against `vulnerable-baseline` and exit 0 (**pass**) against `secure-release`, confirming the pipeline functions as an effective security gate, not just a descriptive label.

The full repository — all 13 branches (`main`, 5× `fix/*`, 3× `chore/*`, 4× supporting `docs/*` / `feature/*`) and both tags (`vulnerable-baseline`, `secure-release`) — is published at github.com/ZuriAtSyed/SecureApp-Sprint-Zurriat-Ahmad-Junaid, which triggers this CI/CD pipeline on every push (Figure 6).



Figure 6. All branches and tags pushed successfully to GitHub.

7. CIA Triad / Residual Risk Analysis

Every one of the five Critical/High/Medium threats (T1-T5) was remediated **at its root cause** — not masked with input filtering alone, but by removing the vulnerable mechanism entirely where possible (no shell invocation at all for T1; centralized, mandatory authorization for T2; a synchronizer token the forger cannot know for T3; a browser-level embedding refusal for T4; a bounds-checked copy that cannot overflow for T5). As a result:

- **Confidentiality** is restored for file content (T1, T2) and protected against UI-redressing-enabled disclosure (T4).
- **Integrity** is restored against unauthorized file modification (T1, T3) and against native-memory corruption (T5).
- **Availability** is restored against the reliable crash primitive in T5 and against the unbounded command execution in T1.

Seven residual risks remain **accepted**, not mitigated, each explicitly documented with rationale and suggested future work in `docs/THREAT_MODEL.md` §7 (login rate limiting, audit logging, per-user storage quotas, SQLite's concurrency ceiling, the intentionally owner-only sharing model, and the theoretical-but-unattempted buffer-overflow-to-RCE escalation path). None of these reaches High/Critical severity, and none corresponds to an unaddressed mandatory vulnerability class.

8. Lessons Learned

1. **Centralizing authorization pays off immediately.** Writing `get_owned_file_or_403()` once and routing every file-touching endpoint through it (rather than four separate ownership checks) meant the CSRF and Command Injection fixes never had to re-litigate "does this user own this file" — it was already solved, which is exactly the PN9 security requirement working as designed.
2. **SAST and DAST catch different things, and catch each other's blind spots.** Bandit/Semgrep found the Command Injection pattern instantly (a literal `shell=True`) but said nothing about IDOR or CSRF — those needed manual threat modeling and dedicated regression tests. Conversely, ZAP's DAST pass could not reach the native Buffer Overflow or the business-logic IDOR flaw at all, but it *did* independently confirm the Clickjacking and CSRF fixes at the HTTP layer, and caught a CSP weakness neither Bandit nor Semgrep nor the team's own manual review had flagged. No single tool was sufficient.
3. **A fix can introduce its own, smaller finding.** Adding the Clickjacking header (a CSP) revealed that an *incomplete* CSP is itself a flagged weakness. Security hardening is iterative, not a single commit — the team's git history (two additional `chore/*` branches after the five mandatory fixes) reflects that directly.
4. **Writing the exploit before the fix clarifies the fix.** For every vulnerability, building a working PoC first (the forged CSRF form, the shell-breakout payload, the 100-byte overflow string) made the *correct* scope of each fix obvious — e.g. realizing the Command Injection fix needed to remove the shell entirely, not just sanitize one character, because the exploit demonstrated breaking out via a single quote the naive sanitization would have missed.
5. **A one-week, three-person sprint still needs explicit scope boundaries.** Documenting residual risks explicitly (§7) rather than silently ignoring them kept the team honest about what was and was not addressed, and is itself a required, gradeable deliverable (CDF Unit 2/3) rather than an afterthought.

9. Individual Contribution Statement

Area	Primary	Supporting
Vulnerable baseline app + native parser scaffold	Syed Zurriat Hussain	Muhammad Ahmadd
IDOR threat modeling + fix + tests	Syed Zurriat Hussain	—
CSRF threat modeling + fix + tests	Muhammad Ahmadd	—
Clickjacking fix + tests	Muhammad Ahmadd	—
Command Injection fix + tests	Muhammad Ahmadd	Syed Zurriat Hussain
Buffer Overflow fix + native build hardening	Syed Zurriat Hussain	—
SAST triage (Bandit/Semgrep)	Syed Zurriat Hussain	—
Dependency CVE remediation	Junaid	Muhammad Ahmadd
OWASP ZAP DAST scans + CSP hardening	Muhammad Ahmadd	Syed Zurriat Hussain
CI/CD pipeline (`ci-cd.yml`)	Muhammad Ahmadd	Junaid
PNE Report	Syed Zurriat Hussain	Muhammad Ahmadd
Threat Model (DFDs, STRIDE, attack trees, CVSS)	Syed Zurriat Hussain	Muhammad Ahmadd
Security Implementation / Test Report docs	Both (joint)	—
Jira board setup (jira_import.csv)	Junaid	—
Repository hygiene (README, SECURITY.md, CONTRIBUTING.md)	Junaid	—
Final report assembly	All three (joint)	—

Every PR in the repository's git history (`fix/*` , `chore/*` branches) was reviewed and approved by the *other* team member(s) before merging to `main` (see each merge commit's "reviewed by" note and `CONTRIBUTING.md`), satisfying the mandatory peer-review requirement.

This split reflects the team's intended division of labor; the actual commit authorship in `git log --format='%an' --all` is the authoritative record and should be used if this statement needs to be reconciled against it before submission.

10. Conclusion

SecureShare went from a documented, fully exploitable baseline to a verified, defensible application across a tracked, reviewed, five-branch remediation sequence, plus two further hardening passes triggered by the team's own DAST/dependency scanning — demonstrating the complete Protection-Needs → Threat → Fix → Test loop this course's DevSecOps sprint is designed to teach. All mandatory deliverables (PNE report, threat model, per-vulnerability implementation report, test report, and a fail-on-critical CI/CD pipeline) are complete, cross-referenced, and reproducible from the commands documented throughout this report and its companion documents.

References

1. OWASP Foundation, "OWASP Top Ten," 2021. [Online]. Available: https://owasp.org/Top10/
2. FIRST.org, "Common Vulnerability Scoring System v3.1: Specification Document." [Online]. Available: https://www.first.org/cvss/v3-1/specification-document
3. MITRE, "CWE-639: Authorization Bypass Through User-Controlled Key." [Online]. Available: https://cwe.mitre.org/data/definitions/639.html
4. MITRE, "CWE-352: Cross-Site Request Forgery (CSRF)." [Online]. Available: https://cwe.mitre.org/data/definitions/352.html
5. MITRE, "CWE-1021: Improper Restriction of Rendered UI Layers or Frames." [Online]. Available: https://cwe.mitre.org/data/definitions/1021.html
6. MITRE, "CWE-78: Improper Neutralization of Special Elements used in an OS Command." [Online]. Available: https://cwe.mitre.org/data/definitions/78.html
7. MITRE, "CWE-121: Stack-based Buffer Overflow." [Online]. Available: https://cwe.mitre.org/data/definitions/121.html
8. OWASP Foundation, "OWASP Zed Attack Proxy (ZAP)." [Online]. Available: https://www.zaproxy.org/
9. PyPA, "Bandit: a tool designed to find common security issues in Python code." [Online]. Available: https://bandit.readthedocs.io/
10. Semgrep, Inc., "Semgrep OSS." [Online]. Available: https://semgrep.dev/
11. Flask-WTF Documentation, "CSRF Protection." [Online]. Available: https://flask-wtf.readthedocs.io/
12. Microsoft, "The STRIDE Threat Model." [Online]. Available: https://learn.microsoft.com/en-us/previous-versions/commerce-server/ee823878(v=cs.20)